

PROJECT DECK



Sentinel AI

AI-Powered Industrial Safety

A full-stack industrial operations platform that combines live monitoring, rule-based risk scoring, knowledge graph reasoning, and a grounded AI copilot.

82

API endpoints discovered

14

Role-aware frontend
workspaces

4

Dataset groups in
repository

Team Details

Shubhankar Pandey and Meera Singh

Why this project matters



Operational visibility

Dashboards, entity modules, alerts, and live risk feeds bring plant state into one console.



Decision intelligence

Rule engine, similarity search, and knowledge graph analysis turn signals into explainable risk context.



Grounded AI guidance

The copilot pulls citations from indexed content and blends them with live operational evidence before invoking Gemini.



Enterprise-ready backend

FastAPI, async SQLAlchemy, Alembic migrations, PostgreSQL support, and JWT authentication back the product surface.

Source analyzed from committed repository content

Industrial safety teams need context, not disconnected alerts

The codebase is built around plants, zones, equipment, sensors, workers, permits, maintenance, incidents, and risk events because isolated signals are not enough.



What the source code tells us

The repository models every major operational safety entity, which implies that risk emerges from relationships, not single readings.

The frontend ships dedicated workspaces for dashboarding, risk, incidents, permits, maintenance, compliance, notifications, and AI assistance.

The backend includes graph, risk, analytics, and copilot routes because decision speed depends on unified context.

Existing workflows create blind spots and slow response loops

The product structure suggests the team is solving repeated operational gaps across monitoring, triage, coordination, explainability, and compliance.



Operational data lives in silos

Sensors, equipment, workers, permits, and incidents are modeled separately, which means a team needs orchestration to reason across them in time.



Response often starts after escalation

The presence of risk history, live risk feeds, and simulation flows points to a need for earlier detection rather than post-incident reporting alone.



Compliance evidence is expensive to assemble

Compliance reports depend on open incidents, overdue maintenance, and live risks, so manual reporting would be fragmented and slow.

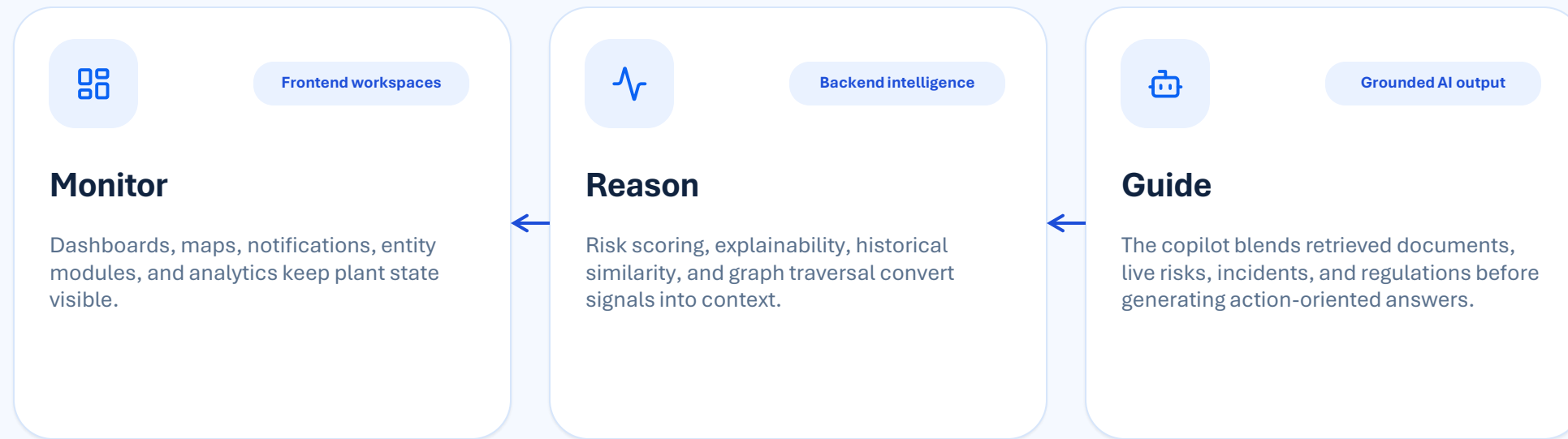


Generic AI is not enough for plant operations

The copilot schema expects citations, regulations, recommendations, and retrieved documents, which means answers must be grounded and auditable.

Sentinel AI unifies monitoring, reasoning, and guided action

The codebase combines a role-aware frontend, a domain-rich FastAPI backend, a rule and graph intelligence layer, and a citation-based AI assistant.



The result is not just visibility. It is a decision layer that can explain risk, surface relevant history, and suggest next actions.

That is the core value proposition implied by the backend services, schemas, and frontend workspaces.

The platform objectives align operations, prediction, and response

These objectives are derived from the service layer, route surface, and frontend workflows present in the repository.



Detect risk earlier

Combine live conditions, maintenance state, permits, workers, and weather-related context before a serious incident occurs.



Prioritize the right actions

Convert risk signals into severity, confidence, explanations, and recommendations that teams can act on immediately.



Preserve operational context

Use graph relationships, incident history, and structured entity data to keep recommendations grounded in plant reality.



Make AI answers traceable

Return citations, regulations, retrieved documents, and saved conversation history instead of opaque text output.

The codebase already spans a full operations command center

Navigation items, route groups, and services indicate a broad feature footprint across plant operations, AI, and compliance.



Live dashboard

Mission control, alerts, recommendations, incidents, and operational KPIs.



Risk center

Risk analysis, history, live feed, confidence, explainability, and actions.



Knowledge graph

Overview, node detail, neighbors, path tracing, and impact analysis.



AI copilot

Grounded chat with citations, regulations, retrieved documents, and memory.



Operational modules

Plants, zones, equipment, workers, permits, maintenance, incidents, and sensors.

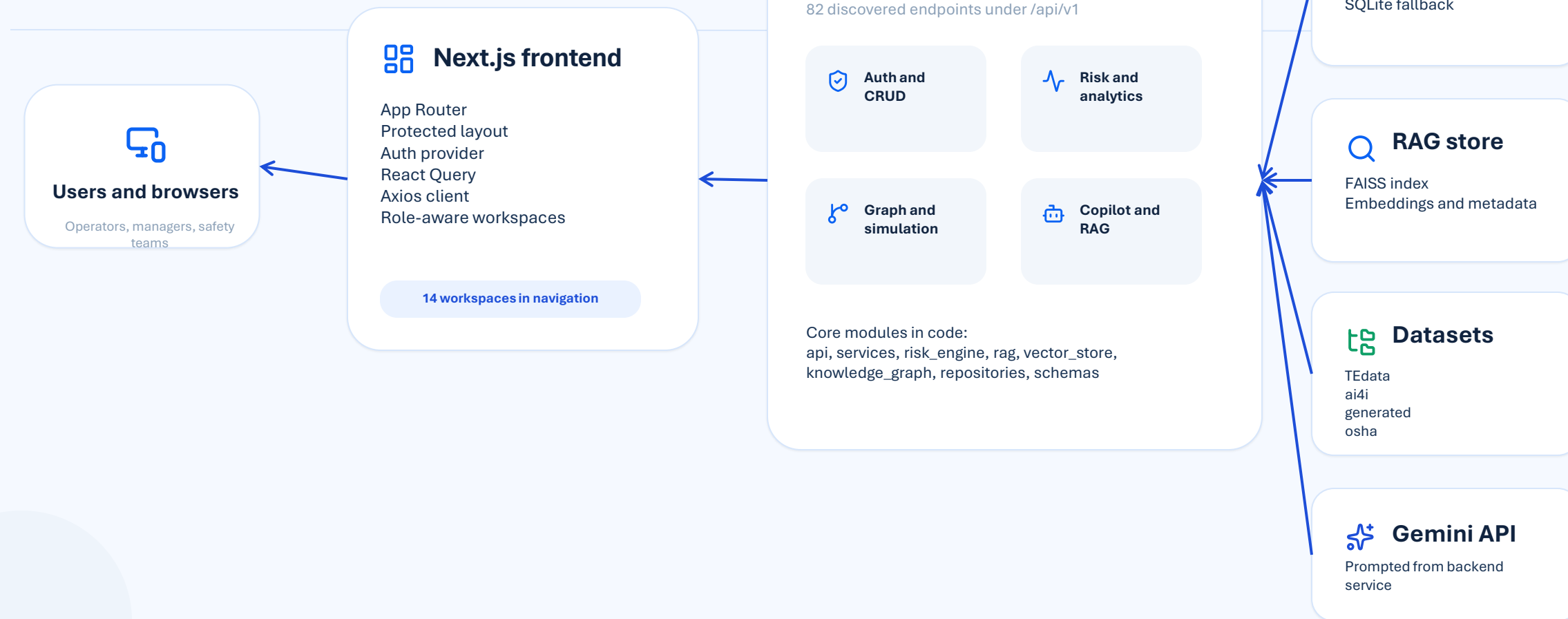


Compliance and simulation

Compliance report generation plus scenario-based safety simulation workflows.

System architecture links browser workflows, backend intelligence, and external AI

Derived from committed frontend, backend, data model, AI modules, and settings.



Stack choices are pragmatic for fast iteration and AI-heavy workflows

The repository favors a modern TypeScript frontend, Python backend services, and explicit AI modules over opaque platform abstractions.



Frontend

Next.js 15, React 19, Tailwind CSS, React Query, Axios, Framer Motion, Recharts, Leaflet.



Backend

FastAPI, Python 3.11 style code, async SQLAlchemy sessions, Pydantic schemas, Alembic migrations.



Database

PostgreSQL-targeted async engine with SQLite fallback for local or emergency execution paths.



AI and ML

SentenceTransformers embeddings, FAISS retrieval, NetworkX graph reasoning, Google Gemini generation.



APIs and integration

REST APIs under /api/v1, JWT auth flows, Axios refresh logic, external Gemini HTTP integration.



Dev and validation

Alembic migration files, dataset loader scripts, backend test modules, and validation utilities.

Observed config surface

DATABASE_URL / SQLITE_FALLBACK_URL

JWT_SECRET / JWT_ALGORITHM

GEMINI_API_KEY / GEMINI_MODEL

RAG_* / DATASET_ROOT / CORS_ORIGINS

The repository separates product surfaces, domain logic, and data assets cleanly

The folder layout shows a clear split between frontend, backend, datasets, and operational scripts.

Top-level repository view

```
Sentinel---AI/  
  backend/  
    app/{api, services, risk_engine, rag, vector_store,  
          knowledge_graph, models, repositories, schemas, tests}  
    alembic/  
    requirements.txt  
  frontend/  
  
src/{app, features, components, providers, services, types, utils}  
package.json  
datasets/{TEdata, ai4i, generated, osha}  
scripts/{load_ai4i, load_osh, load_tennessee, ...}  
.agents/
```

What each root contributes



backend

FastAPI routes, service orchestration, domain models, AI subsystems, repositories, and tests.



frontend

Protected Next.js application with auth, navigation, workspaces, charts, and copilot UI.



datasets and scripts

Tracked industrial datasets and loaders for seeding, ingestion, and data-layer validation.



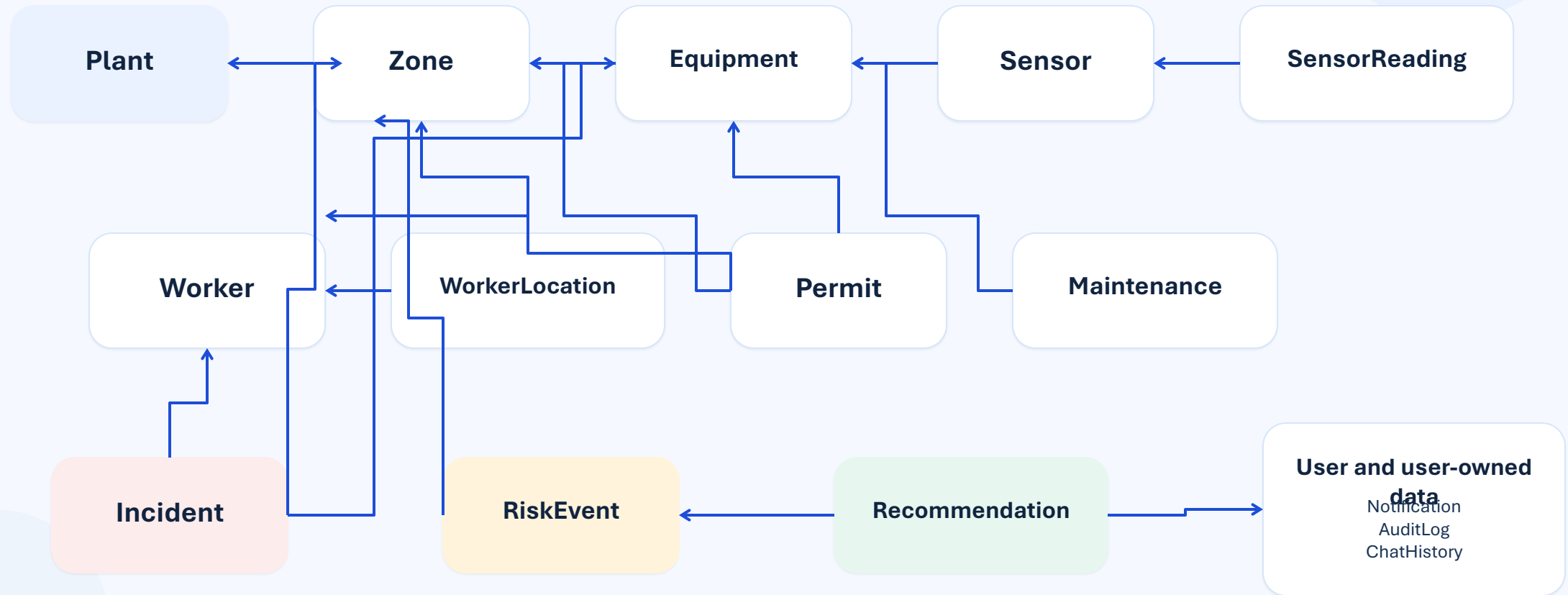
operational maturity signals

6 focused backend test modules and Alembic migrations indicate more than a prototype UI shell.

The data model centers on plants, zones, assets, people, and risk events

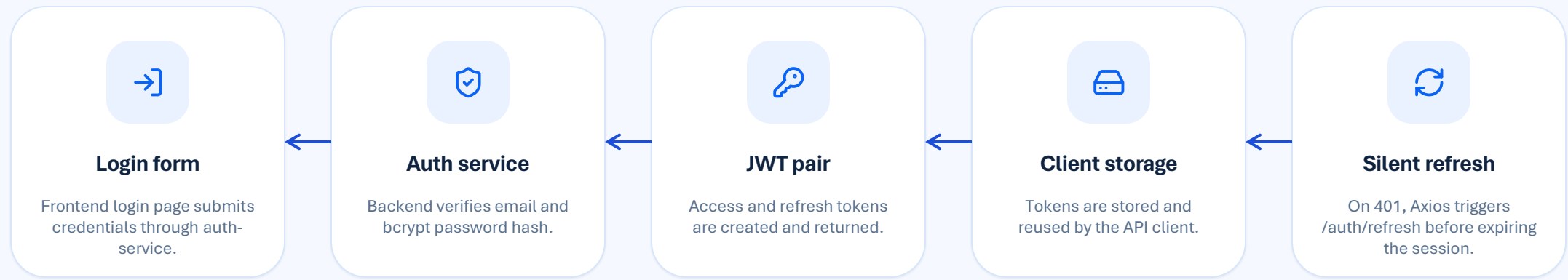
This ER view is simplified from `backend/app/models/entities.py` to keep the main operational relationships readable.

Supporting entities also include ComplianceReport and Document



Authentication is JWT-based with client-side session recovery

The repo contains both backend token issuance and frontend refresh handling, which makes the auth flow presentation-ready and technically concrete.

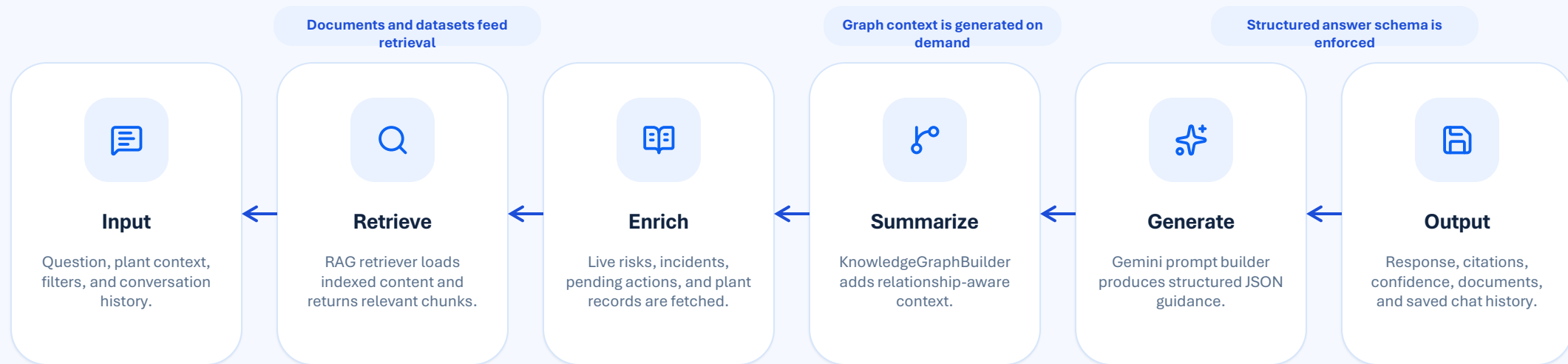


Role guards and route dependencies

Protected routes use `get_current_user` and `require_roles`, while the frontend protected layout redirects unauthenticated users back to `/login`.

The copilot pipeline grounds Gemini with retrieval and live operational context

The pipeline below is derived from `backend/app/services/copilot.py`, `app/ai/gemini.py`, `retriever.py`, `loaders.py`, and the related schemas.



Important implementation detail: the Gemini client also supports a deterministic offline fallback when an API key is absent or a request fails.

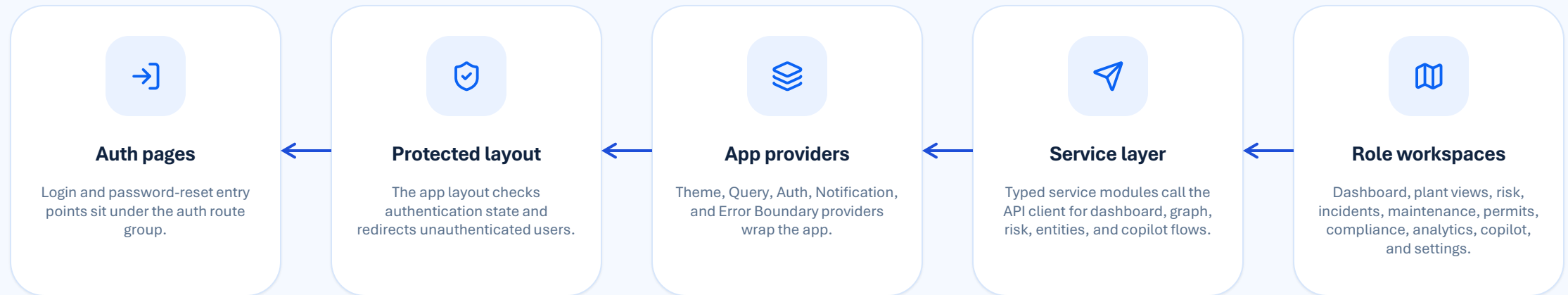
Backend requests move through routing, policy, services, and persistence layers

This view compresses the FastAPI application structure into the main execution flow that most product features follow.



Frontend flow is provider-driven, protected, and role-aware

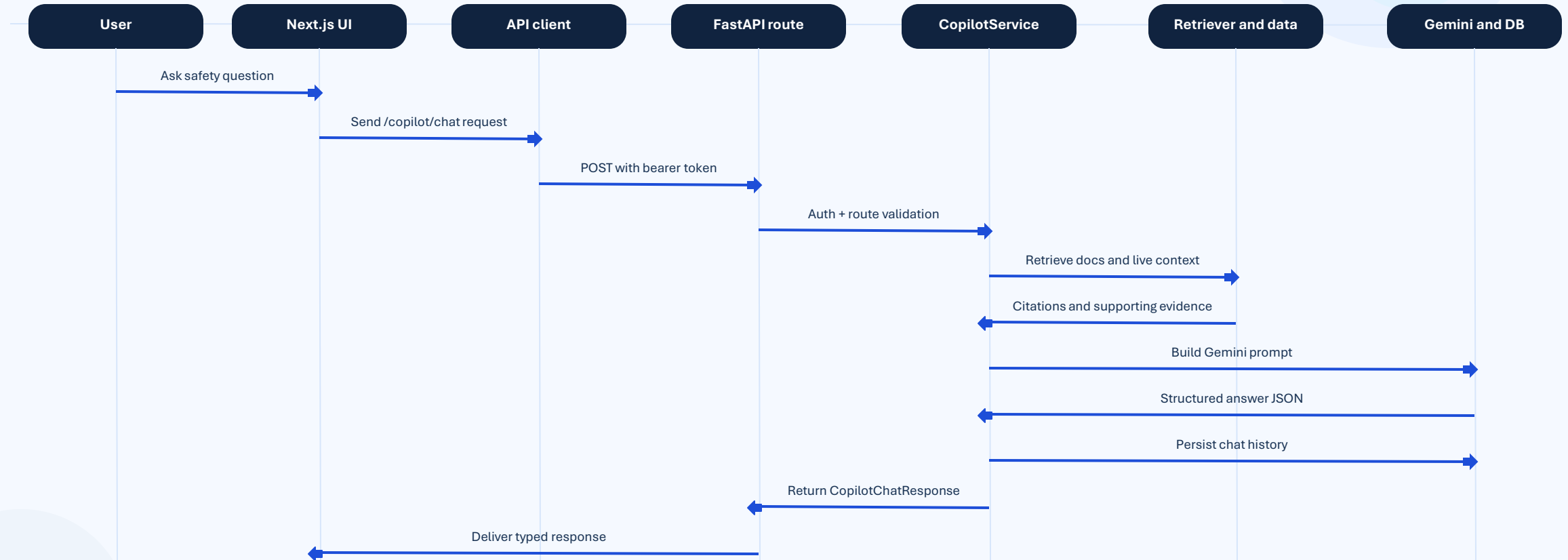
The Next.js application uses a protected app layout, shared providers, and service-based API access to keep feature pages consistent.



User experience detail that matters: the shared Axios client retries safe GETs, refreshes access tokens on 401, and emits a session-expired event if recovery fails.

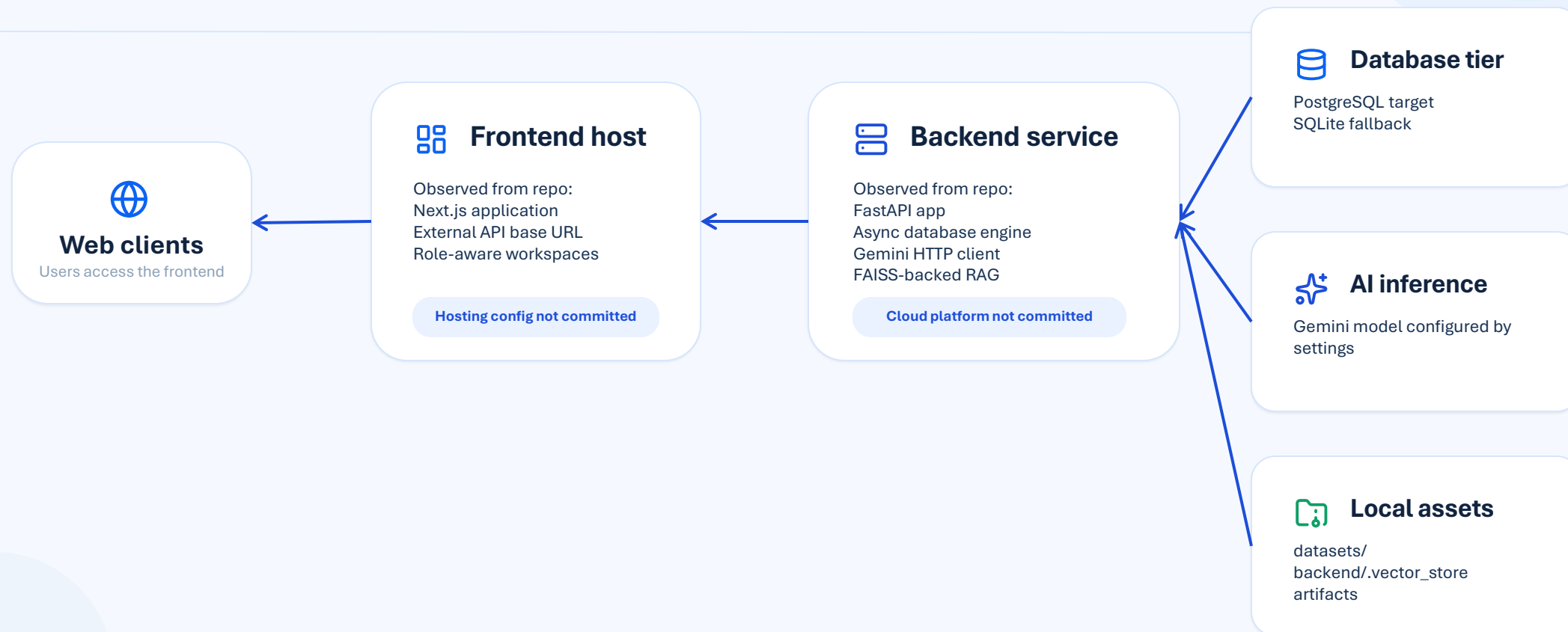
The copilot chat path is the most orchestration-heavy API sequence

This sequence is traced from the committed frontend copilot service, backend route, CopilotService, retriever, Gemini client, and persistence logic.



Deployment topology is partly visible in code, but infrastructure manifests are not committed

This slide separates what is observed from application code and settings from what would need separate deployment artifacts to confirm.



Security is enforced through layered auth, validation, and prompt control

The source code shows a practical security posture spanning authentication, authorization, schema validation, and constrained AI behavior.



JWT session model

Access and refresh tokens are issued server-side and consumed through a protected Axios client.



Role-based access control

`require_roles` and `get_current_user` guard sensitive routes like risk analysis.



Password protection

Passwords are verified with `bcrypt` and never handled as plaintext after receipt.



Typed request validation

FastAPI and Pydantic schemas enforce expected fields and reduce malformed input risk.



Constrained LLM output

Gemini prompting enforces grounded JSON responses and forbids fabricated risk scores.



Boundary configuration

CORS origins, request timeouts, and API settings are centralized in configuration.

The codebase already includes useful optimizations, with clear room to scale further

This slide distinguishes optimizations that are visible today from hotspots that will matter most as usage grows.

Observed in the code



Async database execution

The backend uses an async SQLAlchemy engine with session factories and connection health checks.



Client caching and retry behavior

React Query, safe GET retries, and silent token refresh reduce redundant user-visible churn.



Persistent retrieval substrate

FAISS artifacts and embedding metadata exist on disk instead of requiring pure in-memory recomputation.



Memory-aware document parsing

The loader already uses read-only workbook access and chunk-oriented section extraction patterns.

Scaling pressure points



Copilot request path is orchestration-heavy

Retrieval, live data fetches, graph context, Gemini generation, and persistence all happen in one request cycle.



Frontend expects fast responses

The API client sets a 20 second timeout, so slow backend paths become user-visible quickly.



RAG assets need lifecycle discipline

Datasets, loaders, embeddings, and FAISS indices can become latency and memory hotspots as they grow.



Deployment details are not versioned

Without committed infrastructure manifests, reproducible scaling and performance tuning stay partly manual.

The hardest engineering work sits at the intersection of real-time safety and grounded AI

The repository structure makes the main delivery challenges visible even without runtime logs or product analytics.



Cross-domain context assembly

Plants, zones, equipment, workers, permits, and incidents must all align for risk analysis to be meaningful.



Grounding AI answers

Citations, regulations, retrieved documents, and confidence need to stay aligned with what the system actually knows.



Keeping graph reasoning useful

The graph layer has to stay understandable to operators while still reflecting real relationships and impact paths.



Serving multiple personas

Admins, plant managers, safety officers, maintenance users, and viewers all need different slices of the same system.

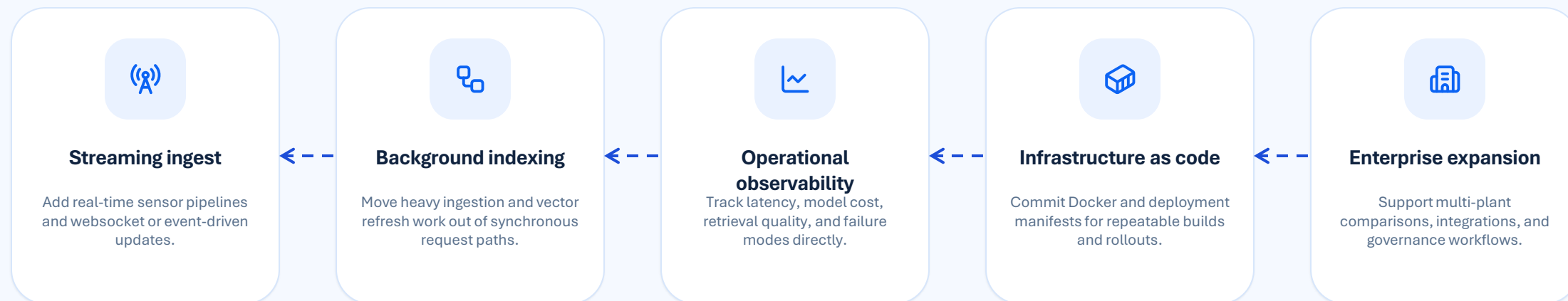


Operationalizing the platform

Application code is strong, but deployment reproducibility and infrastructure visibility still need first-class treatment.

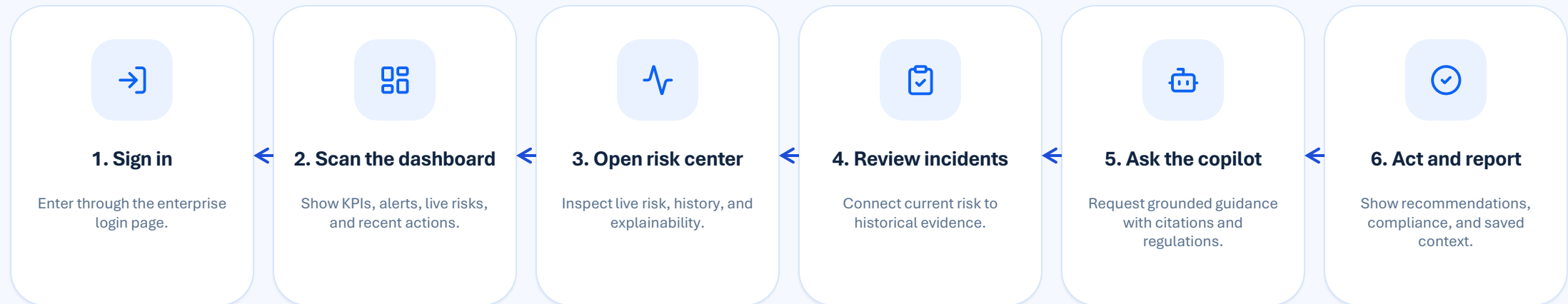
The next growth path is clear: stronger operations, stronger MLOps, stronger delivery maturity

These future directions are derived from current repository capabilities and the gaps that are still visible from source.



A strong demo tells the story from login to insight to action

This sequence is aligned with the implemented frontend and backend flows, so the live demo can follow the actual product surface without improvisation.



THANK YOU

Sentinel AI

AI-Powered Industrial Safety

Questions, feedback, and follow-up discussion

Team details and contact information remain editable on this slide for final delivery.



Built from source-backed analysis

All slide content is tied to committed repository structure and code behavior.



Deck is fully editable

Text, diagrams, and layout elements are native presentation objects rather than flattened screenshots.



Ready for multiple audiences

Suitable for hackathons, placements, technical reviews, and stakeholder demos.

<https://github.com/Shubhankar0126/Sentinel---AI.git>